

COMP3308/3608 Week 6 - Statistical-Based Learning, Evaluating and Comparing Classifiers

Author

K. Yu

2026

Contents

1	Naive Bayes	2
1.1	From Bayes' theorem to a classifier	2
1.2	The weather example (categorical attributes)	3
1.3	A second worked example (volleyball — pure Bayes' theorem)	4
1.4	Issue — zero probabilities and the Laplace correction	5
1.5	Issue — missing values (easy for NB)	5
1.6	Issue — numeric attributes (Gaussian Naive Bayes)	6
1.7	Advantages and disadvantages	7
2	Evaluating and Comparing Classifiers	8
2.1	Accuracy, error rate, and the train/test split	8
2.2	Stratification and repeated holdout	8
2.3	Cross-validation	9
2.4	Comparing two classifiers — the paired t -test	9
2.5	Confusion matrix and beyond-accuracy measures	10
2.6	Cost-sensitive evaluation	11
2.7	Inductive Learning (COMP3608 only)	12

1 Naive Bayes

1.1 From Bayes' theorem to a classifier

For a hypothesis H and evidence E ,

$$P(H | E) = \frac{P(E | H)P(H)}{P(E)}.$$

This is just a rearrangement of the definition of conditional probability, since

$$P(H \cap E) = P(E | H)P(H) = P(H | E)P(E).$$

Quantity	Name
$P(H E)$	posterior of H
$P(H)$	prior of H
$P(E H)$	likelihood
$P(E)$	evidence

The posterior uses *more* information than the prior: it is conditioned on the observed E , whereas the prior is independent of E .

The decision rule

Given the dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, $\mathbf{x}^{(i)} \in \mathbb{R}^d$, $y^{(i)} \in \mathcal{Y} \subseteq \mathbb{R}$, and the set of classes $\mathcal{Y} = \{c_1, \dots, c_K\}$. Classification is to choose the class with the largest posterior. For a test example $\mathbf{x}$,

$$\hat{y} = \operatorname{argmax}_{c \in \mathcal{Y}} P(Y = c | \mathbf{x}) = \operatorname{argmax}_{c \in \mathcal{Y}} \frac{P(\mathbf{x} | Y = c)P(Y = c)}{P(\mathbf{x})}.$$

This is the **maximum a posteriori (MAP)** rule. Because $P(\mathbf{x})$ is the same for every class, it does not affect the argmx:

$$\hat{y} = \operatorname{argmax}_{c \in \mathcal{Y}} P(\mathbf{x} | Y = c)P(Y = c).$$

We only need to compare the **numerators**. (If we want actual posterior *probabilities* rather than just the winning class, we recover $P(\mathbf{x}) = \sum_c P(\mathbf{x} | Y = c)P(Y = c)$ by normalisation.)

The "naive" conditional-independence assumption

The hard term is the joint likelihood $P(\mathbf{x} | Y = c) = P(x_1, \dots, x_d | Y = c)$. Estimating a full joint over d attributes needs exponentially many counts. Naive Bayes makes two simplifying assumptions:

1. **Conditional independence:** given the class, the attributes are independent of one another:

$$P(x_1, \dots, x_d | Y = c) = \prod_{j=1}^d P(x_j | Y = c).$$

2. **Equal importance:** all attributes contribute to the decision (none is privileged the way 1R privileges a single attribute).

Both are *unrealistic* (real attributes are correlated and equally informative) — hence "naive". But the resulting estimator is simple, needs only $O(Kd)$ parameters instead of exponentially many, and works **surprisingly well** in practice.

Putting it together, the Naive Bayes classifier is

$$\hat{y} = \operatorname{argmax}_{c \in \mathcal{Y}} P(Y = c) \prod_{j=1}^d P(x_j | Y = c).$$

Numerical-stability remark

Multiplying many small probabilities underflows to 0. In practice one maximises the **log-posterior**, turning the product into a sum:

$$\hat{y} = \operatorname{argmax}_{c \in \mathcal{Y}} \left[\log P(Y = c) + \sum_{j=1}^d \log P(x_j | Y = c) \right].$$

The argmax is unchanged because log is monotonic.

1.2 The weather example (categorical attributes)

Task: classify the new day

$$\mathbf{x} = \begin{bmatrix} \text{outlook} \\ \text{temp} \\ \text{humidity} \\ \text{windy} \end{bmatrix} = \begin{bmatrix} \text{sunny} \\ \text{cool} \\ \text{high} \\ \text{true} \end{bmatrix}$$

Split the evidence into the four attribute-level pieces $E_1, \dots, E_4$. Using the independence assumption,

$$P(\mathbf{x} | Y = c) = \prod_{j=1}^4 P(E_j | Y = c), \quad c \in \{\text{yes}, \text{no}\}.$$

Estimating the probabilities (the training phase)

Probabilities are estimated from the data by **counting** (maximum likelihood estimates):

$$\hat{P}(x_j | Y = c) = \frac{\#\{\text{examples with attribute } j = x_j \text{ and class } c\}}{\#\{\text{examples with class } c\}}, \quad \hat{P}(Y = c) = \frac{n_c}{n}.$$

	outlook		temperature		humidity		windy		play				
	yes	no		yes	no		yes	no	yes	no			
sunny	2	3	hot	2	2	high	3	4	false	6	2	9	5
overcast	4	0	mild	4	2	normal	6	1	true	3	3		
rainy	3	2	cool	3	1								
sunny	2/9	3/5	hot	2/9	2/5	high	3/9	4/5	false	6/9	2/5	9/14	5/14
overcast	4/9	0/5	mild	4/9	2/5	normal	6/9	1/5	true	3/9	3/5		
rainy	3/9	2/5	cool	3/9	1/5								

outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

proportions of days when play is yes

proportions of days when humidity is normal and play is yes i.e. the probability of humidity to be normal given that play=yes

sydney.edu.au COMP3308/3608 AI, week 6a, 2026 15

Figure 1: Counts-and-probabilities summary table.

The relevant per-attribute estimates for class **yes** (denominator 9, since 9 "yes" days):

$$P(\text{sunny} | \text{yes}) = \frac{2}{9}, \quad P(\text{cool} | \text{yes}) = \frac{3}{9}, \quad P(\text{high} | \text{yes}) = \frac{3}{9}, \quad P(\text{true} | \text{yes}) = \frac{3}{9},$$

and the prior $P(\text{yes}) = 9/14$.

Final calculation

$$P(\text{yes} | \mathbf{x}) \propto \frac{2}{9} \cdot \frac{3}{9} \cdot \frac{3}{9} \cdot \frac{3}{9} \cdot \frac{9}{14} \approx 0.0053,$$

$$P(\text{no} | \mathbf{x}) \propto \frac{3}{5} \cdot \frac{1}{5} \cdot \frac{4}{5} \cdot \frac{3}{5} \cdot \frac{5}{14} \approx 0.0206.$$

(The $\propto$ hides the common factor $1/P(\mathbf{x})$.) Since $0.0206 > 0.0053$,

$$P(\text{no} | \mathbf{x}) > P(\text{yes} | \mathbf{x}) \implies \hat{y} = \text{no}.$$

If we wanted calibrated posteriors, normalise:

$$P(\text{yes} | \mathbf{x}) = \frac{0.0053}{0.0053 + 0.0206} \approx 20.5\%$$

1.3 A second worked example (volleyball — pure Bayes' theorem)

Teams A, B given $P(\text{win} = A) = 0.65$, $P(\text{win} = B) = 0.35$, $P(\text{host} = B | \text{win} = A) = 0.30$, $P(\text{host} = B | \text{win} = B) = 0.75$. Team B hosts the next match — who is favoured?

$$P(\text{win} = A | \text{host} = B) \propto 0.30 \cdot 0.65 = 0.195,$$

$$P(\text{win} = B | \text{host} = B) \propto 0.75 \cdot 0.35 = 0.2625.$$

Compare numerators (shared $P(\text{host} = B)$ cancels): $0.2625 > 0.195 \implies$ **NB predicts B** . Home advantage outweighs A 's higher base win rate.

1.4 Issue — zero probabilities and the Laplace correction

The problem. Suppose in the training data `outlook=sunny` *never* occurred with `play=yes`. Then $\hat{P}(\text{sunny} | \text{yes}) = 0/9 = 0$, and because the likelihood is a **product**,

$$P(\text{yes} | \mathbf{x}) \propto 0 \cdot \dots = 0$$

regardless of how strongly the other three attributes favour *yes*. A single unseen value vetoes the whole class. This is fragile and clearly undesirable.

Laplace correction (add-one smoothing). Add 1 to every numerator and k to the denominator, where k is the number of values that attribute can take:

$$\hat{P}(x_j = v | Y = c) = \frac{n_{v,c} + 1}{n_c + k}.$$

For `outlook` ($k = 3$ values: sunny/overcast/rainy) within the 9 "yes" days:

$$P(\text{sunny} | \text{yes}) = \frac{0 + 1}{9 + 3} = \frac{1}{12}, \quad P(\text{overcast} | \text{yes}) = \frac{4 + 1}{9 + 3} = \frac{5}{12}, \quad P(\text{rainy} | \text{yes}) = \frac{3 + 1}{9 + 3} = \frac{4}{12}.$$

The three still sum to 1. No value gets probability 0, yet for large n_c the change is negligible. Intuition: pretend you saw k extra examples, one for each value.

Generalisation — the m -estimate. Add mp_i to the numerator and m to the denominator:

$$\hat{P}(x_j = v_i | Y = c) = \frac{n_{v,c} + mp_i}{n_c + m}, \quad \sum_i p_i = 1,$$

where p_i is a *prior* on attribute value v_i and m controls how much that prior counts relative to the data.

- **Large m :** trust the prior, distrust sparse data. **Small m :** let the data dominate.
- Choosing $p_i = 1/k$ (equiprobable values) and $m = k$ recovers Laplace exactly: $\frac{n+1}{n_c+k}$.
- *Advantage:* principled; *disadvantage:* must estimate the p_i .

1.5 Issue — missing values (easy for NB)

Missing value in the test example. Simply *drop* the attribute from the product. E.g. for `outlook=?, temp=cool, humidity=high, windy=true`:

$$P(\text{yes} | \mathbf{x}) \propto \frac{3}{9} \cdot \frac{3}{9} \cdot \frac{3}{9} \cdot \frac{9}{14} \approx 0.0238, \quad P(\text{no} | \mathbf{x}) \propto \frac{1}{5} \cdot \frac{4}{5} \cdot \frac{3}{5} \cdot \frac{5}{14} \approx 0.0343.$$

The numbers are *larger* than before (one fewer factor < 1), but the comparison is still **fair** because the same factor is omitted from both classes — the missing piece divides out.

Missing value in a training example. Do not include it in the counts: estimate each $\hat{P}(x_j | c)$ from the examples where that attribute is actually present (use the per-attribute observed count as the denominator, not n_c).

1.6 Issue — numeric attributes (Gaussian Naive Bayes)

Counting does not work for continuous attributes (e.g. `temperature=66`; $P(\text{temp} = 66)$ exactly is 0). **Assume each numeric attribute is Gaussian within each class** and use its probability density function (pdf):

$$f_X(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp \left[-\frac{(x - \mu)^2}{2\sigma^2} \right].$$

Estimate, per class c , the sample mean and **sample** standard deviation (note the $n - 1$ denominator — Bessel's correction):

$$\mu = \frac{1}{n_c} \sum_i x_i, \quad \sigma = \sqrt{\frac{1}{n_c - 1} \sum_i (x_i - \mu)^2}.$$

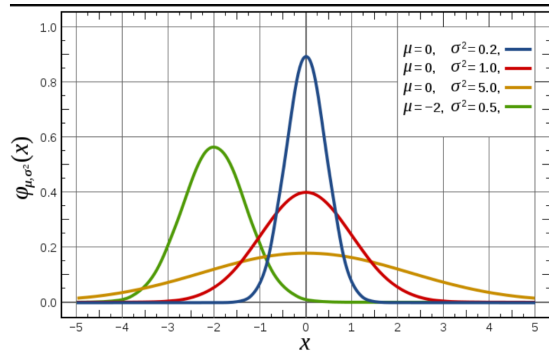


Figure 2: The Gaussian pdf can be visualised as a bell curve.

Density vs probability. $f_X(x)$ is *not* a probability — it is a density. The probability that value lies in a tiny interval $(x - \varepsilon/2, x + \varepsilon/2)$ is approximately $\varepsilon f_X(x)$. Since the same ε appears for every class, it cancels in the comparison, so we may plug the density values directly into the Naive Bayes product.

Worked example. For class "yes", temperature has $\mu = 73$, $\sigma = 6.2$:

$$f_X(\text{temp} = 66 | \text{yes}) = \frac{1}{6.2\sqrt{2\pi}} \exp \left[-\frac{(66 - 73)^2}{2 \cdot 6.2^2} \right] \approx 0.034.$$

With $f(\text{humidity} = 90 | \text{yes}) \approx 0.0221$:

$$P(\text{yes} | \mathbf{x}) \propto \frac{2}{9} \cdot 0.034 \cdot 0.0221 \cdot \frac{3}{9} \cdot \frac{9}{14} \approx 0.000036,$$

$$P(\text{no} | \mathbf{x}) \propto \frac{3}{5} \cdot 0.0291 \cdot 0.038 \cdot \frac{3}{5} \cdot \frac{5}{14} \approx 0.000136,$$

giving $P(\text{no} | \mathbf{x}) > P(\text{yes} | \mathbf{x}) \implies$ no play (same verdict as the categorical version).

1.7 Advantages and disadvantages

Advantages

- Simple; probabilities computed directly with the independence assumption.
- Clear probabilistic semantics for representing, using and learning knowledge.
- Excellent computational complexity: **one pass** over the data to gather all statistics. $O(np)$ -style cost (one scan, $p = \text{\#examples}$, attributes k -valued). Works for both nominal and (Gaussian) continuous attributes.
- Often beats more sophisticated methods $\implies$ *always try the simple method first*.
- Robust to isolated noise (noise is averaged out when estimating the conditional probabilities).

Disadvantages

- **Correlated attributes** violate the independence assumption and erode performance (double-counting evidence). *Remedy*: feature selection to drop redundant attributes.
- **Gaussian assumption** for numeric attributes often fails. *Remedies*: discretise numeric $\rightarrow$ nominal; use a different pdf (Poisson, binomial, gamma, etc.); transform toward Gaussian; or use **kernel density estimation** (assumes no fixed parametric form).

2 Evaluating and Comparing Classifiers

Goal of learning: a classifier that **generalises** — classifies correctly on *new* data unseen during training. Everything below is about *estimating* that generalisation ability.

2.1 Accuracy, error rate, and the train/test split

For a labelled evaluation set,

$$\text{Accuracy} = \frac{\text{\#correctly classified}}{\text{\#total}}, \quad \text{Error Rate} = 1 - \text{Accuracy}.$$

They sum to 1 (or 100%).

Holdout procedure. Split $\mathcal{D}$ into two *non-overlapping* sets — training (typically 2/3) and test (1/3). Build the classifier on the training set, measure accuracy on the test set.

- **Accuracy on training data is overly optimistic** — it measures fit, not generalisation, and can be inflated by memorisation. Always quote **test** accuracy.

Validation set (a third set). Some methods have two stages — (1) build the model, (2) tune hyperparameters. The **golden rule: the test data must never be used in any way to build/tune the classifier**. So use three disjoint sets:

1. **Training** — build the model.
2. **Validation** — tune parameters (e.g. decision-tree pruning, neural-net early stopping).
3. **Test** — final, untouched accuracy estimate.

Making the most of the data. Larger training set $\rightarrow$ better classifier; larger test set $\rightarrow$ better accuracy *estimate*; these compete. Standard resolution: after evaluation is finished, **retrain the final deliverable on all the data (train+val+test joined)** — but still quote the accuracy that was measured on the held-out test data.

2.2 Stratification and repeated holdout

Problem with a single random holdout: the split may be unrepresentative (e.g. a class under- or entirely absent from training).

Stratification. Sample so that each class appears in (roughly) the same proportion in every subset as in the full data. Used *together with* holdout or cross-validation.

Repeated holdout. Repeat the holdout (say) 10 times with different random splits (optionally stratified) and **average** the accuracies $\rightarrow$ more reliable estimate. Weakness: the test sets across repetitions can **overlap**, which biases the estimate. Removing overlap leads to cross-validation.

2.3 Cross-validation

K -fold cross-validation

1. Split $\mathcal{D}$ into K equal-sized, non-overlapping folds.
2. For each fold: train on the other $K - 1$ folds, test on the held-out fold, record accuracy Acc_k .
3. Report the average:

$$\text{Acc}_{\text{CV}} = \frac{1}{K} \sum_{k=1}^K \text{Acc}_k.$$

Every example is used for testing exactly once and for training $K - 1$ times; **test sets never overlap**.

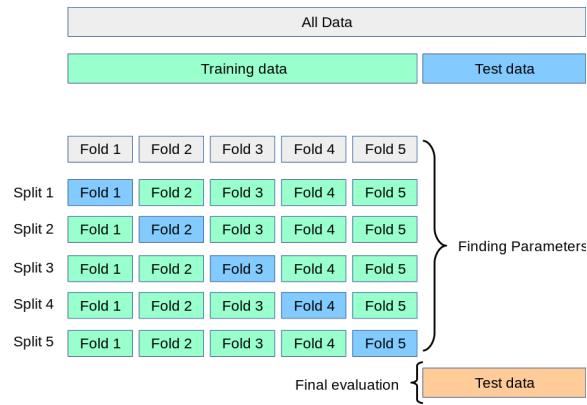


Figure 3: An example of $K = 5$ -fold cross validation.

Leave-one-out CV (LOOCV). The extreme case $K = n$ (so we build the classifier n times).

- *Advantages:* maximal training data each round (best chance of an accurate model); **deterministic** — no random sampling, so no point repeating it (identical result every time).
- *Disadvantage:* cost — n model builds $\rightarrow$ only practical for small datasets.

2.4 Comparing two classifiers — the paired t -test

10-fold CV gives classifiers C_1 and C_2 a mean accuracy each. A raw difference (e.g. 91.3% vs 89.4%) might be **noise**. We test whether the difference is **statistically significant** using a **paired t -test** (paired because both classifiers are evaluated on the *same* folds).

Procedure (over K folds):

1. Per-fold differences $d_i = \text{Acc}_{1,i} - \text{Acc}_{2,i}$, with mean $d_{\text{mean}} = \frac{1}{K} \sum_{i=1}^K d_i$.

2. Sample standard deviation of the differences (an estimate of the true SD):

$$\sigma = \sqrt{\frac{1}{K-1} \sum_{i=1}^K (d_i - d_{\text{mean}})^2}.$$

For K sufficiently large the d_i are approximately Gaussian.

3. Confidence interval, with $t_{1-\alpha}$ read from a t -table at $K-1$ degrees of freedom and confidence level $1-\alpha$:

$$Z = d_{\text{mean}} \pm t_{1-\alpha} \frac{\sigma}{\sqrt{K}}.$$

4. **If the interval contains 0:** difference *not* significant. **If it excludes 0:** significant.

Worked example. $K = 10$, $d_{\text{mean}} = 3.5$, $\sigma/\sqrt{K} = 0.5$, 95% confidence. Degrees of freedom $K-1 = 9 \implies t = 2.26$:

$$Z = 3.5 \pm 2.26 \cdot 0.5 = 3.5 \pm 1.13 = [2.37, 4.63].$$

The interval excludes 0, so the difference **is statistically significant** at 95%.

2.5 Confusion matrix and beyond-accuracy measures

For two classes (positive = yes, negative = no):

	predicted yes	predicted no
actual yes	TP (true positive)	FN (false negative)
actual no	FP (false positive)	TN true negative

Correct predictions lie on the diagonal (TP, TN); **errors** are off-diagonal (FP, FN).

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{FN} + \text{TN}}.$$

For a K -class confusion matrix (e.g. Weka's iris output below), accuracy = sum of diagonal / total:

```
a  b  c  <-- classified as
50  0  0 | a = Iris-setosa
  0 44  6 | b = Iris-versicolor
  0  3 47 | c = Iris-virginica
```

Accuracy = $(50 + 44 + 47)/150 = 141/150 \approx 94\%$.

Precision, recall, F_1

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad R = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad F_1 = \frac{2PR}{P + R}.$$

- **Precision** — of every thing predicted positive, what fraction really is?

- **Recall** — of all actual positives, what fraction did we catch?
- F_1 — the **harmonic** mean of P and R ; high only when *both* are high.

Spam example. TP = 24, FN = 1, FN = 70, TN = 5:

$$P = \frac{24}{24 + 70} = 25.5\%, \quad R = \frac{24}{24 + 1} = 96\%, \quad \text{Accuracy} = \frac{24 + 5}{100} = 29\%.$$

High recall but terrible precision: it catches almost all spam but floods the user with 70 misclassified legitimate emails — a *bad* filter despite high recall.

The precision-recall trade-off (extreme cases).

- *Block everything:* $R = 100\%$, $P = 25\%$ — recall is trivially maxed.
- *Block almost nothing (one email):* $P = 100\%$, $R = 12.5\%$ — precision trivially maxed. We can usually push one up at the expense of the other; F_1 guards against gaming a single number.

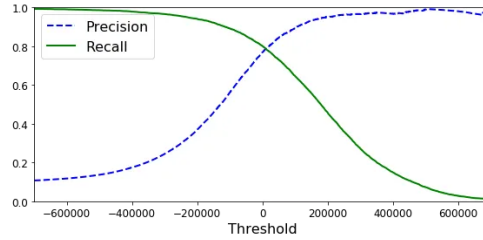
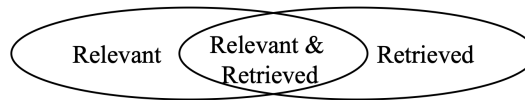


Figure 4: Typical precision-recall pattern.

Example (text retrieval). Given a query (e.g. “Cross validation”), a text retrieval system retrieves a number of documents. Here,



$$\text{Precision} = \frac{|\text{Relevant} \cap \text{Retrieved}|}{|\text{Retrieved}|}, \quad \text{Recall} = \frac{|\text{Relevant} \cap \text{Retrieved}|}{|\text{Relevant}|}.$$

2.6 Cost-sensitive evaluation

Different errors carry different costs, so a single error count is misleading.

- **Spam:** blocking a legitimate email (FP) is usually *much* worse than letting one spam through (FN).

- **Loan approval:** lending to a defaulter $\gg$ refusing a good borrower.
- **Oil-spill detection:** missing a spill $\gg$ a false alarm.
- **Promotional email:** junk to a non-responder $\ll$ failing to mail a would-be responder.

Remedy: weight the errors – e.g. “blocking a legitimate email counts as X errors” ($X = 10, 100, \dots$ depending on the application) – and compute **weighted accuracy** / **precision** / F_1 . The right operating point depends entirely on the relative costs.

2.7 Inductive Learning (COMP3608 only)

Why evaluate empirically at all? Because supervised learning is **inductive** — inferring the general from the particular (“all apples I’ve seen are red $\implies$ all apples are red”). Induction is not deductively valid, so we can only *estimate* how good a hypothesis is, empirically, on fresh data.

Formal setup. Given examples $(\mathbf{x}, f(\mathbf{x}))$ from an unknown target function f , find a hypothesis $h \approx f$ from a **hypothesis space** H (e.g. polynomials of degree $\leq k$). A hypothesis is **consistent** if it fits all training examples.

Which consistent hypothesis is best? Many hypotheses can fit the same points (a line and a wiggly degree-7 polynomial can both pass through 7 points).

- **Ockham’s razor:** prefer the *simplest* hypothesis consistent with the data — it is more likely to capture the real pattern (generalise) and is cheaper to compute. Definition “simplicity” is hard in general; here a low-degree polynomial is clearly simpler.
- A perfect fit to training data is **not** the goal — we want to *extract patterns*, not *memorise* (the degree-7 fit memorises noise $\rightarrow$ poor generalisation = **overfitting**).

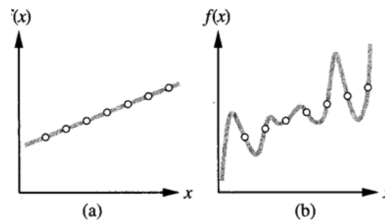


Figure 5: (a) Straight-line fit vs (b) degree-7 wiggle through the same 7 points.

Importance of the hypothesis space. If H does not contain f , you can never learn f exactly (e.g. $\sin x$ is not a polynomial, so no polynomial H recovers it). A problem is **realisable** if H contains the true function — but in practice we cannot check this, since we do not know f (that is what we are trying to learn).

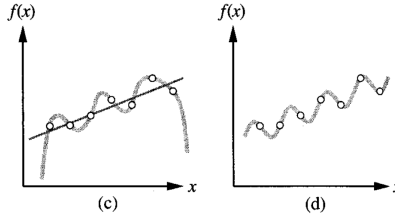


Figure 6: (c) Degree-1 and degree-6 polynomial fits to sinusoidal data vs (d) the true $\sin x$.

Take-away. Empirical evaluation on held-out data is the best available proxy for generalisation precisely because the inductive problem gives us no guarantees — we measure performance on new examples rather than assume it.